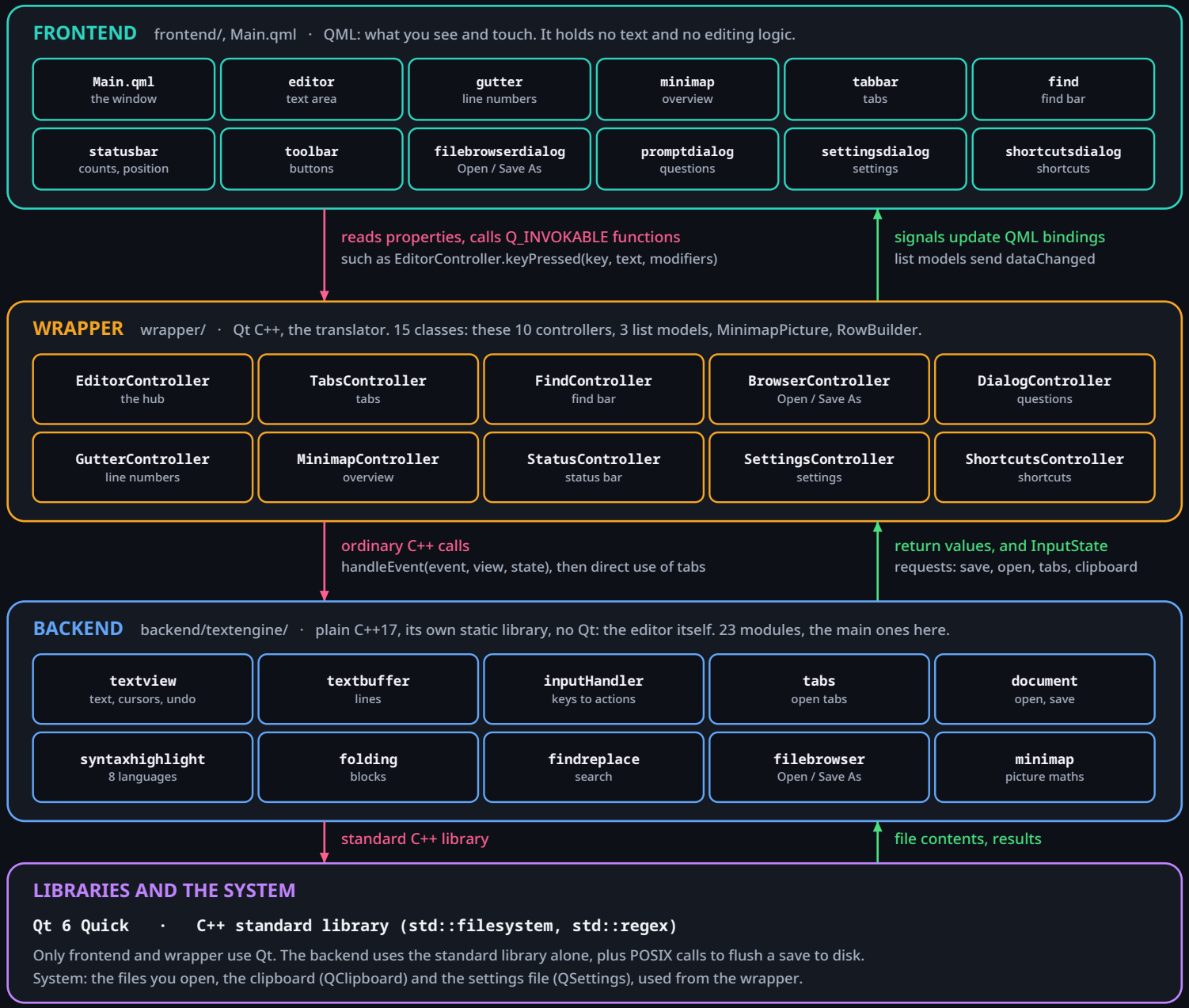
The big picture

Three layers, each talking only to the next one. All of them run on one thread: the app starts no threads of its own.



The rules that keep it clean

The backend knows nothing about Qt.
No backend file includes a Qt header. It also knows no screen: keys, clicks and rows are its own plain types.

QML never reaches the backend.
Every screen goes through a wrapper singleton, such as EditorController.

The engine asks, the wrapper does.
The input handler cannot save or change tabs itself: it adds Requests to InputState, and EditorController carries them out.

One object owns the editor.
EditorController holds the tabs and input state. The other controllers reach them through it.

One thread.
No code starts a thread, so slow work blocks typing. Files over 50 MB are refused.

RowBuilder has no Qt either.
It lives in wrapper/ but is plain C++: Qt classes only copy its results.

What is what

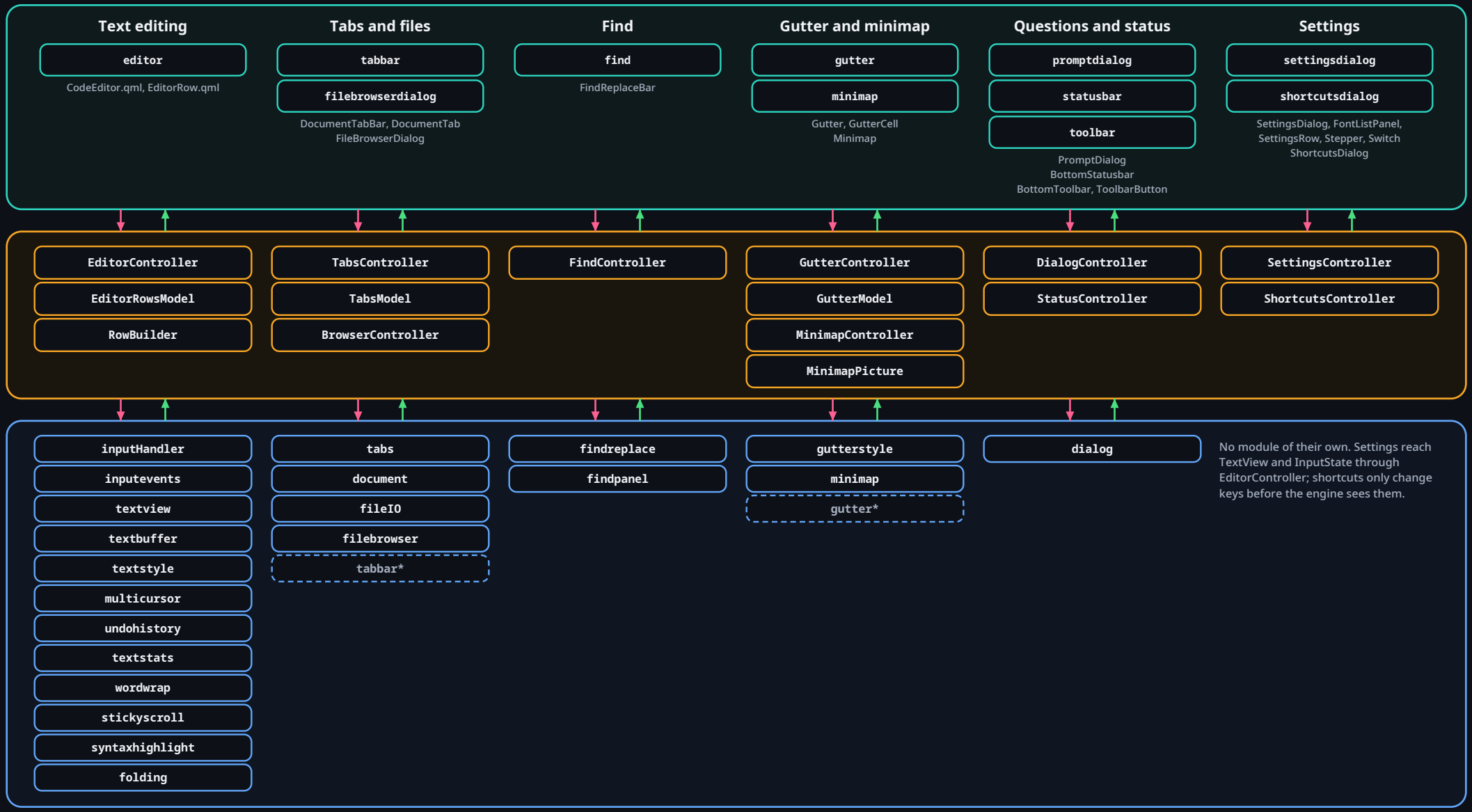
QML	language that describes screens
Controller	C++ class QML can talk to
Singleton	one shared object, used by name
Signal	a class says something changed
Binding	QML value that updates by itself
Q_INVOKABLE	C++ function QML may call
Request	engine asks the program to act

In numbers

QML files (with Main.qml)	**22**
wrapper C++ files, classes	**28, 15**
backend C++ files, modules	**62, 23**

The blueprint

Read each column top to bottom: the screens, the wrapper classes they use, the backend modules behind them. Every part is here once.



calls into the layer below

reports back

name* built but not used by the app: nothing includes gutter.h or tabbar.h.

The columns group parts by subject. Classes also use each other, and shared modules reach across columns: page 3 lists them class by class. Above the columns: Main.qml holds all the screens, and every screen shares Theme.qml and Icon.qml.

The connections

Generated from the code: QML names, and the engine and wrapper names used inside each class, checked against its #include chain.

Wrapper class	Used by these screens	Backend modules it uses	Other wrapper parts it uses
EditorController	editor, find, Main	document, fileIO, folding, gutterstyle, inputevents, inputHandler, multicursor, stickyscroll, syntaxhighlight, tabs, textbuffer, textview	DialogController, EditorRowsModel, RowBuilder, ShortcutsController
EditorRowsModel	editor	tabs	RowBuilder
RowBuilder	none: used from C++ only	findreplace, folding, gutterstyle, multicursor, stickyscroll, syntaxhighlight, tabs, textbuffer, textstyle, textview, wordwrap	-
BrowserController	filebrowserdialog, Main	filebrowser, inputevents, inputHandler	EditorController
DialogController	Main, promptdialog	dialog, document, fileIO, inputevents, inputHandler, tabs, textview	EditorController
FindController	find, Main	findpanel, findreplace, inputHandler, tabs, textview	EditorController
GutterController	gutter	tabs	EditorController, GutterModel, RowBuilder
GutterModel	gutter	tabs	RowBuilder
MinimapController	minimap	gutterstyle, inputHandler, minimap, syntaxhighlight, tabs, textview	EditorController, RowBuilder
MinimapPicture	minimap	minimap	MinimapController
SettingsController	settingsdialog	document, inputHandler, tabs, textview	EditorController
ShortcutsController	shortcutsdialog	-	-
StatusController	Main	document, inputHandler, syntaxhighlight, tabs, textbuffer, textstats, textview	EditorController
TabsController	Main	inputHandler, tabs	DialogController, EditorController, TabsModel
TabsModel	Main	-	-

How to read a row: FindController is used by frontend/find and by Main (the toolbar's Find button). It uses the backend modules named, and works through EditorController. "Main" is Main.qml; the other screen names are folders of frontend/.

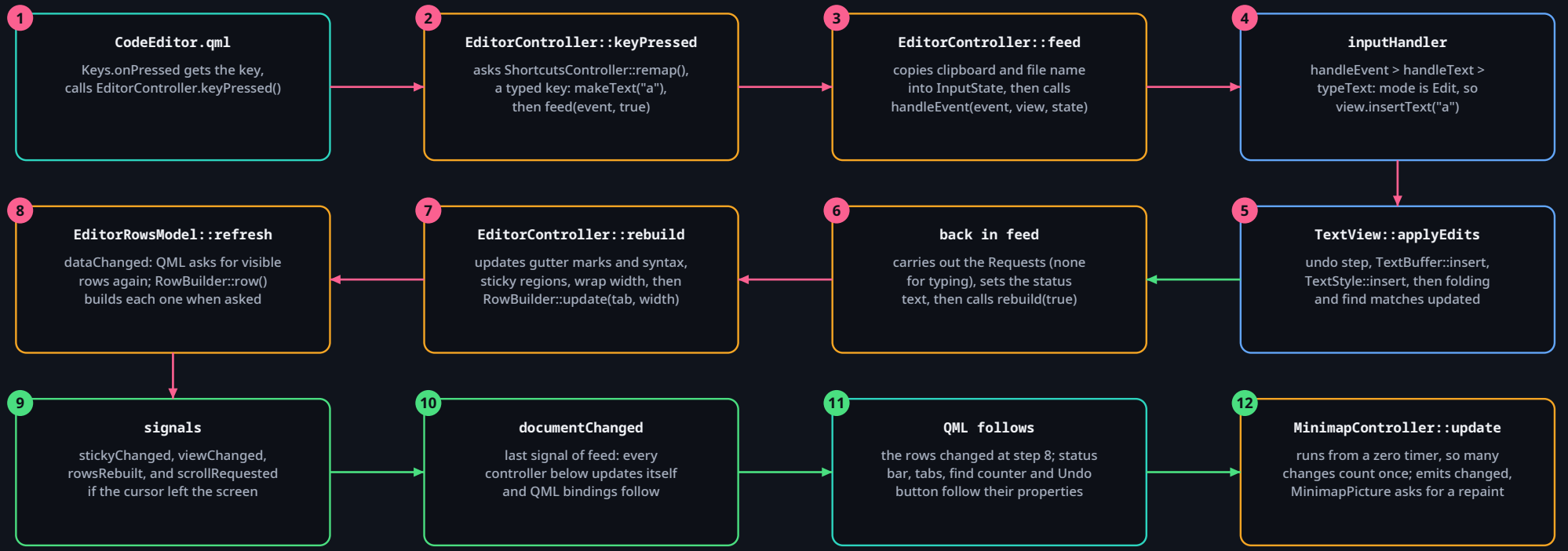
EditorRowsModel, GutterModel and TabsModel have no QML name: they reach QML through EditorController.rows, GutterController.rows and TabsController.tabs. RowBuilder is used by C++ alone. Calls inside the backend are not listed. ShortcutsController is the only controller that uses no other wrapper class and no backend module.

Following one keystroke

You press the letter a in the text. Everything up to the screen update runs on the GUI thread, one step after the other.

GUI THREAD

all of this project's code, in order



QT'S RENDER STEP

Qt's code, not this project's

13 Qt draws the frame. Qt's documentation says MinimapPicture::paint() is called on the renderer thread while the GUI thread is blocked, so it can read the minimap safely. That is the only place another thread could touch this code. In the software test run here, paint() ran on the GUI thread.

Who listens to EditorController

every connect() in the wrapper, plus QML. Nothing calls these one by one: each class connects itself.

documentChanged

TabsController::sync
titles, unsaved dots, current tab

StatusController::sync
position, counts, message

FindController::sync
bar state, counter

BrowserController::sync
file list

documentChanged, continued

DialogController::advance
finishes save, then close or quit

SettingsController
indent for new tabs, Alt+M memory

MinimapController::schedule
repaint later

QML: canUndo, canRedo
Undo and Redo buttons

other signals

rowsRebuilt: GutterController
refresh; Minimap schedule

stickyChanged: GutterController
refreshSticky; CodeEditor.qml's pinned rows

viewChanged: MinimapController
schedule

scrollRequested: CodeEditor.qml
scrolls the list

Starting up

main.cpp is short: it starts Qt and loads Main.qml. The controllers are not made in main.cpp. QML creates each one the first time it uses it.

0 Before main()
each of the 8 language headers registers its LanguageDef (a static variable)

→ **1 QGuiApplication**
Qt starts

→ **2 QQmlApplicationEngine**
if Main.qml fails to load, objectCreationFailed exits with -1

→ **3 loadFromModule(...)**
"ChaoticNotepad", "Main": this creates the window and every screen

During step 3: the order each constructor started, measured

on Qt 6.11.2. Qt decides this order; only the arrows lower on this page are fixed by the code.

1. `MinimapPicture`
2. `MinimapController`
3. `EditorController`
4. `DialogController`
5. `BrowserController`
6. `ShortcutsController`
7. `SettingsController`
8. `StatusController`
9. `FindController`
10. `GutterController`
11. `TabsController`

Models, RowBuilder and the tabs are made inside their owners.

4 Read the command line
each argument that does not start with "-" becomes a file URL, relative names counted from the current folder

→ **5 openUrls(urls)**
only if files were named: EditorController::instance()-> openUrls(urls) opens them in tabs

→ **6 QGuiApplication::exec()**
the event loop: the app runs until you quit.
The window shows by itself: Main.qml says visible: true.

How the controllers depend on each other at start-up

An arrow means "its constructor needs". So EditorController always finishes first, whatever order QML picks.



EditorController's own constructor needs no other controller. It makes one empty tab, the input state, the row builder and the rows model, then calls rebuild(true). In the measured run, MinimapController started first and created it.

ShortcutsController needs nobody: it reads its keys from QSettings. EditorController only reaches it on the first key press.

Each controller makes the link itself, in its constructor, so main.cpp needs to know only EditorController.

Building, packaging, releasing

One platform in practice: Linux x86_64. Three ways from source to a running app, all starting from CMakeLists.txt.

CMakeLists.txt

textengine
static library: every .cpp of backend/textengine, no Qt

appChaoticNotepad
the program: main.cpp

QML module ChaoticNotepad
Main.qml, frontend/, wrapper/, the icon font

links
Qt6::Quick and textengine

C++17, Qt 6.10 or newer, version 2.0

install.sh
sudo: from source, into /usr/local

cmake Release, build in build/
cmake --install
refreshes desktop and icon caches

packaging/build-appimage.sh
on your computer, with a version

builds in build-release/, installs to AppDir/
linuxdeploy collects Qt, then the original
libraries are put back; writes dist/

.github/workflows/release.yml
on GitHub, for a pushed v* tag

Ubuntu 22.04, Qt 6.10.0
runs build-appimage.sh, smoke test
publishes the release

Making a release

ordinary pushes build nothing: only a v* tag starts this

git push origin v2.0
a tag, not a commit

→ **install system packages**
and Qt 6.10.0

→ **work out the version**
v2.0 gives 2.0

→ **build-appimage.sh**
AppImage, tar.xz, checksums

→ **smoke test**
run 8 s under xvfb

→ **upload run files**
kept with the run

→ **publish release**
only for a tag

The smoke test does not stop a release: it is marked continue-on-error, so after a crash at start-up the run goes on and publishing still happens. "Run workflow" on the Actions page builds and keeps the files, and publishes nothing.

What a release contains

ChaoticNotepad-2.0-x86_64.AppImage
one file with Qt inside: run it

ChaoticNotepad-2.0-linux-x86_64.tar.xz
the app and scripts: install-app.sh, no root, in your home

SHA256SUMS.txt
checksums of the two files above

How each install is removed

install.sh
sudo ./uninstall.sh removes it from /usr/local

install-app.sh
./uninstall-app.sh removes it from your home; documents and settings stay

AppImage
nothing is installed: delete the file

Platforms and tools

The release workflow and packaging scripts are Linux only, and there are no per-platform source files. CMakeLists.txt keeps Qt's macOS and Windows settings from the project template, and fileio.cpp has Unix-only code to flush a save to disk, but no Windows or macOS build is made or tested here.

Why the script puts libraries back: the AppImage tools rewrite the headers of what they bundle, which crashed the app at start-up on Fedora 44. AppRun then sets the library paths itself.

build-appimage.sh needs Qt 6 with qmake, cmake, curl and a C++17 compiler. It downloads linuxdeploy, its Qt plugin and appimagetool once, into tools/. Generated folders: build-release/, AppDir/, dist/.